

Hardware Architecture Section Preview

Abstract?This standalone file previews only the hardware architecture section for the MXFP4 Gated DeltaNet FPGA accelerator manuscript.

IV. Hardware Architecture

A. Overview

The proposed architecture implements one Gated DeltaNet decode layer for batch-one autoregressive inference. For each token step t , the kernel receives token-dependent vectors

$$q_t, k_t, v_t \in \mathbb{R}^d$$

and scalar control values

$$\beta_t, g_t \in \mathbb{R}.$$

The kernel updates a recurrent state matrix

$$S_t \in \mathbb{R}^{d \times d}$$

and produces an output vector

$$o_t \in \mathbb{R}^d.$$

The design focuses on the decode primitive after the query, key, and value projections have already been generated. The main architectural goal is to keep S_t resident in on-chip FPGA memory across token steps.

B. Gated DeltaNet Recurrence

The hardware follows the Gated DeltaNet update rule used by the software reference model. First, the previous state is queried by the key vector:

$$r_t = S_{t-1} k_t$$

where r_t is the predicted value retrieved from the old state. The prediction error is then computed as

$$e_t = r_t \odot v_t.$$

The state is updated using a beta-scaled rank-one correction:

$$S_t = S_{t-1} + \beta_t e_t k_t^T.$$

Finally, the output vector is produced from the updated state:

$$o_t = S_t q_t.$$

Depending on matrix layout, the implementation may use the transposed form $S_t^T q_t$ or $q_t S_t$, but the hardware operation is still a state-vector product.

C. Persistent State Storage

The recurrent state matrix has fixed size $d \times d$ and does not grow with sequence length. For a model with H heads, the total stored state is

$$H d^2 \text{ elements.}$$

This fixed-size property allows the state to be stored in partitioned on-chip memory banks. The proposed design keeps S_t persistent across token steps. Therefore, the top-level interface only transfers the current token inputs and the output vector. It does not transfer the full recurrent state every token. This changes the main decode bottleneck from external state movement to on-chip state update bandwidth.

D. Five-Phase Token Dataflow

Each token is processed using a fixed five-phase schedule:

$$\text{prepare} \rightarrow \text{read} \rightarrow \text{update} \rightarrow \text{writeback} \rightarrow \text{output}.$$

The dominant operations are the state-vector product and the rank-one state update. With key-direction parallelism P_K , a full pass over the $d \times d$ state costs approximately

$$T_{\text{pass}} \approx d^2 / P_K \text{ cycles.}$$

Since the design requires one main state read phase and one writeback/update phase, the token latency can be approximated as

$$T_{\text{token}} \approx 2d^2 / P_K + O(d).$$

The lower-order $O(d)$ terms come from vector operations such as beta scaling, residual computation, and output correction.

E. Native MXFP4 Datapath

The accelerator uses MXFP4 operands in the low-precision computation path. Each block of B elements shares one E8M0 scale, while each element is represented as a four-bit E2M1 value. A block can be written as

$$x_i \approx 2^{s_b} x_i', \quad i \approx b,$$

where s_b is the shared block exponent and x_i' is the E2M1 element value. This representation reduces storage and arithmetic cost while preserving local dynamic range. Unlike a dequantized design, the proposed datapath does not convert MXFP4 operands into BF16 or FP32 before multiplication.

F. LUT-Based E2M1 Multiplication

For two E2M1 values a and b , the multiplier computes

$$p = a \times b$$

directly using LUT logic. Each operand is decoded into sign, exponent, and mantissa fields:

$$a = (-1)^{s_a} 2^{e_a} m_a, \quad b = (-1)^{s_b} 2^{e_b} m_b.$$

The product sign is

$$s_p = s_a \oplus s_b,$$

the exponent path performs

$$e_p = e_a + e_b \oplus \text{bias},$$

and the mantissa path computes

$$m_p = m_a \times m_b.$$

The result is normalized and converted into a fixed-point partial product. This keeps the inner multiply small enough to map efficiently into FPGA LUTs.

G. Block-Exponent Alignment

Because MXFP4 uses shared block scales, products from different blocks may have different effective exponents. If two input blocks have exponents s_a and s_b , then the product block has effective exponent

$$s_{\text{prod}} = s_a + s_b.$$

Before accumulation, the partial product is shifted according to this exponent:

$$p_i' = p_i \ll s_i,$$

where s_i aligns the product to the accumulator scale. The aligned partial products are accumulated into a fixed-point accumulator:

$$A = \sum_i p_i'.$$

This stage preserves the block-scale information without widening the inner multiply to BF16 or FP32.

H. Parallelism Parameters

The architecture is controlled by three main compile-time parameters:

$$B, \quad P_K, \quad P_V.$$

Here, B is the MXFP4 block size, P_K is the key-direction parallelism, and P_V is the value-direction parallelism. B determines how many E2M1 values share one block exponent. P_K controls how many state columns are processed in parallel. P_V controls how many value or output lanes are updated together.

$$B = 32, \quad P_K = 16, \quad P_V = 8.$$

These parameters determine the tradeoff between latency, resource utilization, and scale-alignment overhead.

I. Summary

The architecture maps the Gated DeltaNet recurrence onto a persistent-state FPGA datapath. The recurrent state $S_t \in \mathbb{R}^{d \times d}$ is stored on chip, token updates are scheduled as fixed read-update-write phases, and MXFP4 arithmetic is implemented natively through LUT-based E2M1 multiplication and block-exponent-aware accumulation. This design targets the two main costs of decode: moving recurrent state and performing repeated state-vector products.